CloudGuard + GRC Merge Security Review

Date: 2026-08-29

Scope: the newly merged `grc-platform/` module, its CloudGuard connector, the CloudGuard connector endpoint, and the GRC production container/dependency configuration. This is a source review and dependency audit, not a penetration test of a deployed environment.

Executive Summary

The GRC code builds, but it is **not ready for a public production deployment**. The principal blocker is authentication: 94 of 98 imported GRC controllers use `DevAuthGuard`. Under the production compose configuration that guard rejects every request. Enabling its override to restore access would instead authenticate callers as a seeded administrator. The new CloudGuard connector controller uses the same guard.

Tenant segregation also fails open. A shared connector token can select any CloudGuard evidence owner through a request header, while missing or malformed GRC tenant mappings fall back to the shared `anonymous` owner. This can return another tenant's evidence.

The dependency audit reported 38 packages with advisories: 3 low, 21 moderate, 13 high, and 1 critical. The critical Vitest advisory is development-only and Vitest is not copied into the frontend runtime image. A production-only npm audit still reports 32 affected packages: 1 low, 21 moderate, and 10 high. At least the Multer upload denial-of-service issue is directly relevant to exposed GRC upload endpoints. The exact deployed dependency tree is currently non-reproducible because service Dockerfiles ignore the repository lockfile and run `npm install` independently.

Findings

GRC-AUTH-001: Production authentication is either unavailable or bypassed

- Severity: **Critical**
- Location:
  - `grc-platform/services/controls/src/cloudguard/cloudguard.controller.ts:9`
  - `grc-platform/services/shared/src/auth/dev-auth.guard.ts:201-213`
  - `grc-platform/services/shared/src/auth/dev-auth.guard.ts:230-239`
  - `grc-platform/docker-compose.prod.yml:393-418`
- Evidence: The connector controller uses `@UseGuards(DevAuthGuard)`. The guard throws unless `NODE_ENV` is development/test or `USE_DEV_AUTH=true`. Production compose sets `NODE_ENV=production` and does not provide `USE_DEV_AUTH`. If the override is enabled, the guard creates a seeded admin user without validating the caller's bearer token. The same development guard appears on 94 of 98 GRC controllers.
- Impact: In the documented production configuration, GRC API requests fail, including `/api/cloudguard/dashboard`. Setting `USE_DEV_AUTH=true` as a workaround creates

an authentication bypass and gives callers seeded administrator privileges and organization context.
- Fix: Replace `DevAuthGuard` with the existing `JwtAuthGuard` or a properly validated Keycloak/OIDC guard on all production controllers. Prefer a global deny-by-default production guard with explicit public-route metadata. Keep `DevAuthGuard` available only in development/test builds and make startup fail if it is enabled with `NODE_ENV=production`.
- Mitigation: Until the guard migration is complete, do not expose the GRC services publicly. Network isolation is not a substitute for fixing the guard, but it limits immediate exposure.
- False positive notes: None. The production environment and guard behavior are explicit in source.

GRC-TENANT-001: Connector tenant resolution fails open and permits cross-tenant selection

- Severity: **High**
- Location:
  - `grc-platform/services/controls/src/cloudguard/cloudguard.service.ts:70-84`
  - `backend/main.py:484-485`
  - `backend/main.py:4684-4696`
- Evidence: The GRC service maps an organization to a CloudGuard user, but absent entries and invalid JSON fall back to `CLOUDGUARD_USER_ID`, which defaults to `anonymous`. CloudGuard accepts `x-cloudguard-user` from any request holding the one shared connector token and uses it directly as the evidence owner.
- Impact: An unmapped organization can receive shared anonymous evidence. Compromise or misuse of the GRC connector service can select another CloudGuard evidence owner and read that tenant's compliance summary and artifact metadata. This violates the intended `tenant_id` segregation model.
- Fix: Bind connector authorization to a tenant claim. Use a short-lived signed service token containing an allowed `tenant_id`, resolve the CloudGuard owner server-side, and reject headers that attempt to override it. Replace the environment JSON map with a validated database mapping keyed by immutable GRC organization ID and CloudGuard tenant ID. Fail closed when no mapping exists.
- Mitigation: Use a separate connector credential per tenant and restrict each credential to one CloudGuard tenant until claim-scoped service authentication is implemented.
- False positive notes: A fully trusted connector service may legitimately broker data, but the current single secret has unrestricted tenant scope and therefore an unnecessarily large blast radius.

CG-CONNECTOR-AUTH-001: The CloudGuard connector endpoint can fail open and accepts secrets in URLs

- Severity: **High**
- Location:

  - backend/main.py:467-481
  - backend/main.py:4684-4696
- Evidence: `_is_connector_request_authorized()` authorizes requests when no connector token is configured unless `CONNECTOR_AUTH_REQUIRED` is explicitly `true`. `_request_token()` also accepts `connector_token` from the query string.
- Impact: A deployment that omits both settings exposes compliance data, evidence metadata, and worker status without authentication. Query-string tokens can leak through proxy logs, browser history, monitoring systems, and referrer data.
- Fix: Make the connector fail closed unconditionally. Require a connector signing key or service credential during startup, accept it only from an authorization header, compare secrets with `secrets.compare_digest`, and reject startup if connector auth is incomplete.
- Mitigation: The Dokploy stack already sets `CONNECTOR_AUTH_REQUIRED=true` and requires `CONNECTOR_TOKEN`; preserve that configuration and verify it in every other deployment path.
- False positive notes: The Dokploy configuration is protected, but the endpoint itself and other compose/development paths retain insecure defaults.

GRC-DEPS-001: Known vulnerabilities remain in runtime dependencies

- Severity: **High**
- Location:
  - grc-platform/services/controls/package.json:30-57
  - grc-platform/services/shared/package.json:13-28
  - grc-platform/frontend/package.json:26-58
  - grc-platform/package-lock.json
- Evidence: `npm audit --omit=dev` reports 32 affected packages: 1 low, 21 moderate, and 10 high. The locked tree includes `@nestjs/platform-express@11.1.21`, `multer@2.1.1`, `axios@1.16.1`, `js-yaml@4.1.1`, `nodemailer@8.0.7`, `sanitize-html@2.17.4`, `dompurify@3.4.3`, and `react-router-dom@6.30.3`. GRC exposes multiple Multer-backed upload endpoints, making the deeply nested field-name and aborted-upload denial-of-service advisories relevant. The sanitizer, HTTP client, email, and router packages are also used by runtime code.
- Impact: Authenticated or exposed upload paths may be used to exhaust process resources. Other advisories may enable request manipulation, unsafe URL handling, XSS bypasses under affected configurations, or server-side file/URL access if vulnerable library features are reached.
- Fix: Upgrade direct dependencies to patched releases, regenerate a reviewed lockfile, and run unit/integration tests before deployment. Prioritize Multer/Nest platform, Axios, Nodemailer, React Router, DOMPurify, and sanitize-html. Do not run `npm audit fix --force` blindly because several suggested remediations are major-version changes.
- Mitigation: Apply reverse-proxy request/body/rate limits to upload routes and keep those routes authenticated while dependency upgrades are tested.
- False positive notes: The original critical advisory is `vitest`, a development-only test

runner, and is not present in the Nginx frontend runtime image. Some production-counted advisories are not exploitable with current options; the Multer issue is directly applicable, while others require feature-specific validation.

GRC-BUILD-001: Production dependency trees are not reproducible and include development packages

- Severity: **Medium**
- Location:
  - `grc-platform/services/controls/Dockerfile:17-20`
  - `grc-platform/services/controls/Dockerfile:58-75`
  - `grc-platform/services/frameworks/Dockerfile:11-14`
  - `grc-platform/services/frameworks/Dockerfile:46-63`
  - `grc-platform/frontend/Dockerfile:13-20`
- Evidence: Service Dockerfiles copy package manifests from individual workspaces, but only the repository root has the workspace lockfile. Each image runs `npm install` rather than `npm ci`. Backend production stages then copy the complete builder `node_modules`, including development dependencies.
- Impact: Two builds from the same commit can contain different dependency versions. The repository audit does not reliably describe the deployed image, and unnecessary compilers/test/build packages expand image size and attack surface.
- Fix: Build from the workspace root with the committed lockfile and `npm ci`. Prune to production dependencies before copying the runtime tree, or create a clean production install using the same lockfile. Generate an SBOM and scan the final images in CI.
- Mitigation: Pin image digests and retain generated SBOMs for each deployment until the Dockerfiles are corrected.
- False positive notes: `--ignore-scripts` use reduces install-script risk but does not provide reproducibility or remove development dependencies.

GRC-AUTH-002: Browser bearer tokens remain accessible to JavaScript

- Severity: **Medium**
- Location:
  - `grc-platform/frontend/src/lib/secureStorage.ts:99-141`
  - `grc-platform/frontend/src/contexts/AuthContext.tsx:124-131`
  - `grc-platform/frontend/src/lib/api/client.ts:94-118`
  - `grc-platform/frontend/nginx.conf:492-495`
- Evidence: Keycloak access tokens are stored in `sessionStorage` and read by Axios interceptors. The CSP permits `unsafe-eval`, and the application processes uploaded document HTML through third-party parsing/sanitization code.
- Impact: Any successful same-origin XSS can steal the bearer token and use it until expiration. Session storage shortens persistence but does not protect the token from JavaScript execution.
- Fix: Prefer a backend-for-frontend session using `Secure`, `HttpOnly`, `SameSite` cookies, with CSRF protection for state-changing requests. If browser bearer tokens re-

main, keep them only in memory, shorten their lifetime, rotate them, and remove `unsafe-eval` when Monaco configuration allows.
- Mitigation: Patch sanitizer/router dependencies, enforce a tighter CSP, and keep token scopes and lifetimes minimal.
- False positive notes: This is not evidence of an existing XSS exploit. It describes the impact amplification if one occurs.

GRC-TEST-001: The new connector has no authorization or tenant-isolation regression tests

- Severity: **Medium**
- Location:
  - `grc-platform/services/controls/src/cloudguard/cloudguard.service.ts:16-85`
  - `grc-platform/services/controls/src/cloudguard/cloudguard.controller.ts:7-18`
  - `backend/main.py:4684-4696`
- Evidence: No tests reference `CloudGuardService`, `CloudGuardController`, `/api/connectors/grc/dashboard`, `CLOUDGUARD_TENANT_USER_MAP`, or `x-cloudguard-user`.
- Impact: The most important security contract in the merge, namely authenticated tenant-scoped evidence access, can regress without detection. Current build checks prove compilation, not authorization behavior.
- Fix: Add tests for missing/invalid connector credentials, valid tenant-scoped access, unmapped tenants, malformed mappings, cross-tenant attempts, token rotation, CloudGuard outage/timeout, and evidence-limit boundaries. Add a production-compose smoke test using real JWT validation rather than development auth.
- Mitigation: Treat the connector as internal-only and manually verify tenant boundaries before each deployment until automated tests exist.
- False positive notes: None found in the repository search.

Positive Controls Observed

- The GRC browser does not receive the CloudGuard connector secret; the server-side controls service makes the CloudGuard request.
- The connector request has a 15-second timeout and returns generic upstream errors.
- CloudGuard limits evidence results to at most 200 records.
- GRC services use validation pipes, Helmet, rate limiting, non-root runtime users, and container hardening options.
- The frontend production image contains only built static assets, so Vite/Vitest are builder or test concerns rather than browser runtime packages.

Recommended Release Gate

1. Replace development guards with validated production JWT/OIDC authentication.
2. Enforce tenant-scoped connector credentials and fail-closed mappings.
3. Make CloudGuard connector authentication mandatory in code, not only compose configuration.

4. Patch and lock production dependencies, then scan final container images.
5. Add tenant-isolation and production-compose integration tests.

Do not expose the merged GRC publicly until items 1-3 are complete.